# IMovies – Functional Specification Document

## 1. Introduction

- **Project title:** IMovies – Movie Ticket Booking Platform (MERN stack)
- **Team members:** SWUID2026403439

## 2. Project Overview

**Purpose.** IMovies lets customers find a movie, pick exact seats for a show and pay online, while admins run the catalogue, theatres, show schedule and bookings from a separate panel.

**Customer features**

- Register, log in, edit profile and change password
- Browse, search and filter movies by genre, language, now showing / coming soon, with sorting
- Movie page with cast, rating, reviews and showtimes by city and date
- Interactive seat map (max 10 seats, live availability) with a 10-minute seat hold and countdown
- Simulated payment by card, UPI, net banking or wallet
- E-ticket with booking ID, printable; booking history; cancellation up to 2 hours before the show with simulated refund
- Star ratings and reviews; favourites and watchlist (email when a watchlisted movie gets a show)

**Admin features** (at `/admin/login`)

- Dashboard: revenue, tickets sold, top movies, last 7 days
- Manage movies, theatres and screens (custom seat layouts)
- Schedule shows with per-seat-type pricing and overlap protection
- View and cancel all bookings; activate, deactivate or delete users

## 3. Architecture

**Frontend (React 19 + Vite).** A single-page app in plain JavaScript/JSX using React Router v6. `App.jsx` defines the routes; `RequireAuth` guards customer pages (`role="user"`) and the admin area (`role="admin"`). Login state lives in `AuthContext`, API calls go through `api.js`, and shared UI sits in `components/` (Navbar, Footer, MovieCard, Poster, ui.jsx). The Vite dev server proxies `/api` to port 5000, so the browser sees one origin and the login cookie works without CORS setup.

**Backend (Node.js + Express 4).** `server.js` configures the app (helmet, CORS, JSON parser, cookie-parser, morgan, rate limiting on login/register) and mounts eight route groups under `/api`: auth, admin, movies, theatres, shows, bookings, reviews and favourites. Controllers hold the logic; middleware handles authentication, role checks, ObjectId validation and centralised errors. `utils/` holds seat logic, token generation, the simulated payment gateway, email sending and the seed script.

**Database (MongoDB with Mongoose 8).** Eight collections:

| Collection | Key fields | Relations |
|---|---|---|
| User | name, email (unique), hashed password, phone, isActive | – |
| Admin | name, email (unique), hashed password | – |
| Movie | title, description, genre[], language, duration, certificate (U/UA/A/S), cast[], releaseDate, poster, trailerUrl, averageRating, ratingCount, isActive | status (upcoming / now showing) is derived from releaseDate |
| Theatre | name, city, address, screens[] (name + row layout: row, seats, type Regular/Premium/Recliner), isActive | embeds screens |
| Show | screenId, screenName, startTime, endTime, prices per seat type, bookedSeats[], isActive | movie, theatre |
| Booking | bookingCode, seats[] (seatId, type, price), totalAmount, status (pending/confirmed/cancelled/expired), expiresAt, payment (method, status, transactionId, paidAt) | user, show, movie, theatre |
| Review | rating 1–5, comment (max 1000 chars); one per user per movie | user, movie |
| Favourite | type (favourite / watchlist); unique per user, movie and type | user, movie |

## 4. Setup Instructions

**Prerequisites**

- Node.js 18 or newer (check with `node -v`)
- MongoDB: either MongoDB Community Server installed locally, or a free MongoDB Atlas cluster whose connection string goes in `MONGO_URI`

**Installation**

1. Extract the project and open two terminals, one in `server/` and one in `client/`.
2. In `server/`, run `npm install`.
3. Open `server/.env` and set the variables below (change `JWT_SECRET` to a long random string).
4. Run `npm run seed` to create the admin account, sample movies, a theatre and shows (it only seeds an empty database).
5. In `client/`, run `npm install`.

| Variable (server/.env) | Purpose |
|---|---|
| `PORT` | API port (default 5000) |
| `NODE_ENV` | `development` or `production` (production makes cookies `secure`; `sameSite=none`) |

| Variable (server/.env) | Purpose |
|---|---|
| MONGO_URI | MongoDB connection string |
| JWT_SECRET | Secret used to sign tokens |
| JWT_EXPIRES_IN | Token lifetime (default 7d) |
| CLIENT_URL | Allowed CORS origin (default http://localhost:5173) |
| SEAT_HOLD_MINUTES | Seat hold while payment is pending (default 10) |
| ADMIN_NAME, ADMIN_EMAIL, ADMIN_PASSWORD | Default admin created by the seed script |
| SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS, EMAIL_FROM | Optional email settings; if SMTP_HOST is empty, emails print in the server console |

Optional in client/: VITE_API_URL is needed only when the API is hosted on a different origin.

## 5. Folder Structure

**Client (client/)**

```
client/
  index.html, vite.config.js, package.json, .env.example
  public/favicon.svg
  src/
    main.jsx, App.jsx        entry point and routes
    api.js, hooks.js, utils.js
    index.css
    context/AuthContext.jsx  login state
    components/              Navbar, Footer, MovieCard, Poster, ui.jsx
    pages/                   Home, Movies, MovieDetail, SeatSelection, Checkout,
                             BookingDetail, MyBookings, Favourites, Profile, Login, Register
    pages/admin/             AdminLogin, AdminLayout, Dashboard, AdminMovies,
                             AdminTheatres, AdminShows, AdminUsers, AdminBookings
```

**Server (server/)**

```
server/
  index.js      entry point: loads .env, connects DB, starts server
  server.js     Express app, middleware and route mounting
  config/db.js  MongoDB connection
  models/       user, admin, movie, theatre, show, booking, review, favourite
  controllers/  auth, admin, movie, theatre, show, booking, review, favourite
  routes/       one router per controller
  middleware/   authMiddleware, validateObjectId, errorMiddleware
  utils/        seatUtils, generateToken, paymentGateway, sendEmail, seed, AppError, asyncHandler
```

## 6. Running the Application

Start the backend first, then the frontend, each in its own terminal.

| Part | Directory | Command | Address |
|---|---|---|---|
| Backend (development) | server/ | npm run dev (nodemon) | http://localhost:5000 |
| Backend (plain) | server/ | npm start | http://localhost:5000 |
| Frontend (development) | client/ | npm run dev | http://localhost:5173 |

The project uses Vite, so the frontend command is npm run dev rather than npm start. Check the API at /api/health. For production, run npm run build in client/ and serve client/dist from any static host.

| Role | Login URL | Credentials |
|---|---|---|
| Customer | /register | any email and password you create |
| Admin | /admin/login | admin@imovies.com / Admin@123 (seeded demo account) |

## 7. API Documentation

Base path /api. Every response is JSON with a success flag; errors return { "success": false, "message": "..." }. Access: **Public** = no login, **User** = customer login, **Admin** = admin login, **Any** = either.

| Method | Endpoint | Access | Purpose / parameters |
|---|---|---|---|
| GET | /health | Public | API status check |
| POST | /auth/register | Public | Create account: name, email, password, phone |
| POST | /auth/login | Public | Log in: email, password |
| POST | /auth/logout | Public | Clear the login cookie |
| GET | /auth/me | Any | Current account |
| PUT | /auth/me | Any | Update profile / change password |
| POST | /admin/login | Public | Admin login: email, password |
| GET | /admin/dashboard | Admin | Revenue, tickets sold, top movies, last 7 days |
| GET | /admin/users, /admin/users/:id | Admin | List / view users |
| PUT, DELETE | /admin/users/:id | Admin | Activate or deactivate / delete a user |
| GET | /admin/bookings | Admin | All bookings |
| GET | /movies | Public | Query: search, genre, language, status, sort, page, limit |
| GET | /movies/:id | Public | Movie details |

| Method | Endpoint | Access | Purpose / parameters |
|---|---|---|---|
| POST, PUT, DELETE | /movies, /movies/:id | Admin | Create / update / delete a movie |
| GET | /theatres | Public | Query: city |
| GET | /theatres/:id | Public | Theatre with screens |
| POST, PUT, DELETE | /theatres, /theatres/:id | Admin | Manage theatres |
| POST, PUT, DELETE | /theatres/:id/screens[/:screenId] | Admin | Manage screens and seat layouts |
| GET | /shows | Public | Query: movie, theatre, city, date |
| GET | /shows/:id | Public | Show with seat map and availability |
| POST, PUT, DELETE | /shows, /shows/:id | Admin | Schedule / edit / remove shows |
| POST | /bookings | User | Hold seats (status pending): show, seats |
| POST | /bookings/:id/pay | User | Pay: method (card, upi, netbanking, wallet) and details |
| GET | /bookings/my | User | Own bookings |
| GET | /bookings/:id | Any | One booking (owner or admin) |
| PUT | /bookings/:id/cancel | Any | Cancel up to 2 hours before the show |
| GET | /reviews/movie/:movieId | Public | Reviews for a movie |
| POST | /reviews | User | Add review: movie, rating (1–5), comment |
| PUT | /reviews/:id | User | Edit own review |
| DELETE | /reviews/:id | Any | Delete (owner or admin) |
| GET | /favourites | User | Query: type (favourite / watchlist) |
| POST | /favourites | User | Body: movie, type |
| DELETE | /favourites/:movieId | User | Query: type |

**Example: login**

```
POST /api/auth/login
{ "email": "user@example.com", "password": "secret123" }

200 OK
{
  "success": true,
  "token": "<jwt>",
```

```
  "user": { "_id": "...", "name": "Asha", "email": "user@example.com", "role": "user", "phone": "..." }
}
```

**Example: cancel a booking**

```
PUT /api/bookings/:id/cancel

200 OK
{ "success": true, "message": "Booking cancelled. Refund initiated.", "booking": { ... } }

400 Bad Request (less than 2 hours before the show)
{ "success": false, "message": "Bookings can only be cancelled up to 2 hours before the show" }
```

**Booking flow.** POST /bookings holds the seats for 10 minutes; POST /bookings/:id/pay confirms them. If payment is not made in time the booking becomes expired. The demo gateway rejects a card number ending in 0000, requires a 16-digit card number and a name@bank UPI id, and always accepts net banking and wallet.

## 8. Authentication

- **Passwords** are hashed with bcrypt (10 rounds) before saving and are excluded from queries by default.
- **Tokens.** On login or registration the server signs a JWT containing the account id and role (user or admin), valid for 7 days by default (JWT_EXPIRES_IN).
- **Cookie.** The token is sent in an httpOnly cookie named token (secure and sameSite=none in production, sameSite=lax in development), so scripts in the page cannot read it. A Authorization: Bearer <token> header is also accepted.
- **Authorization.** protect verifies the token and loads the account (rejecting deleted or deactivated users); adminOnly and userOnly enforce roles. Admins have their own collection and login endpoint.
- **Frontend.** AuthContext loads the account from /auth/me, and RequireAuth redirects visitors away from pages that need a login or a specific role.
- **Hardening.** helmet headers, CORS limited to CLIENT_URL, and a limit of 100 login/register attempts per IP every 15 minutes.

## 9. User Interface

Screens are listed with their routes. Screenshots or GIFs can be dropped in under each row once the app is running.

| Screen | Route | Access | What it shows |
|---|---|---|---|
| Home | / | Public | Featured and current movies |
| Movies | /movies | Public | Search, filters and sorting with movie cards |
| Movie detail | /movies/:id | Public | Cast, rating, reviews, showtimes by city and date |
| Seat selection | /shows/:id/seats | Public | Seat map with live availability, up to 10 seats |
| Checkout | /checkout/:id | User | Hold countdown and payment method form |

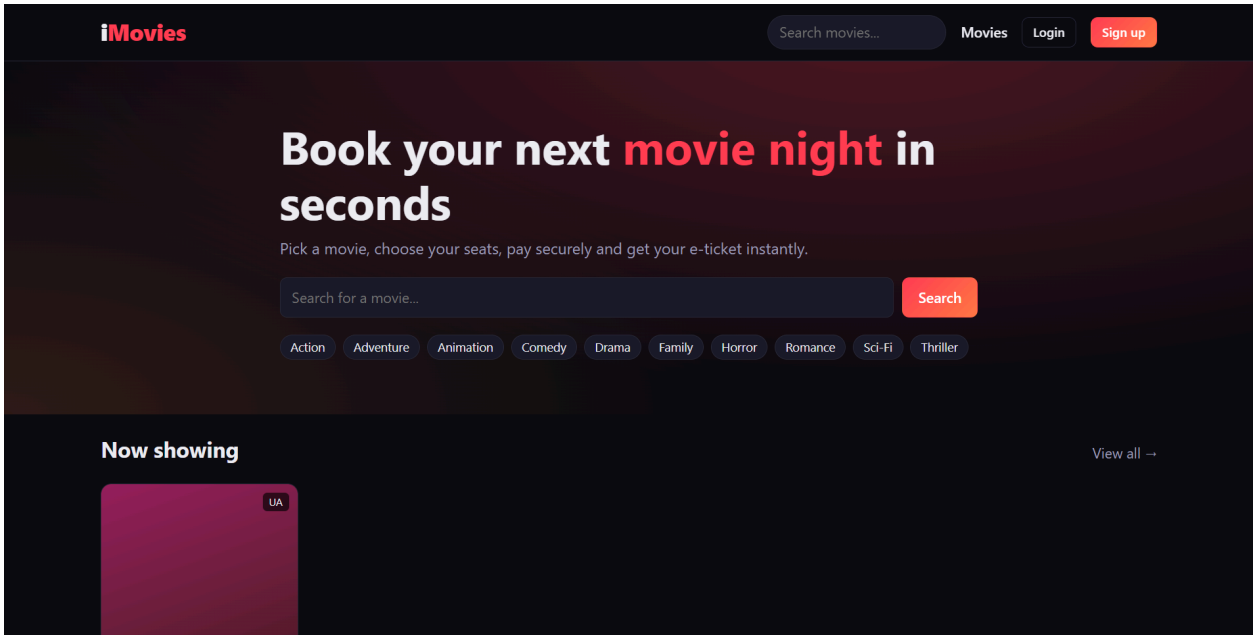
| Screen | Route | Access | What it shows |
|---|---|---|---|
| Booking detail | /bookings/:id | Login | E-ticket with booking ID, printable |
| My bookings | /my-bookings | User | History and cancellation |
| Favourites | /favourites | User | Favourites and watchlist |
| Profile | /profile | User | Edit details, change password |
| Login / Register | /login, /register | Public | Customer authentication |
| Admin login | /admin/login | Public | Admin authentication |
| Admin panel | /admin | Admin | Dashboard, movies, theatres and screens, shows, users, bookings |

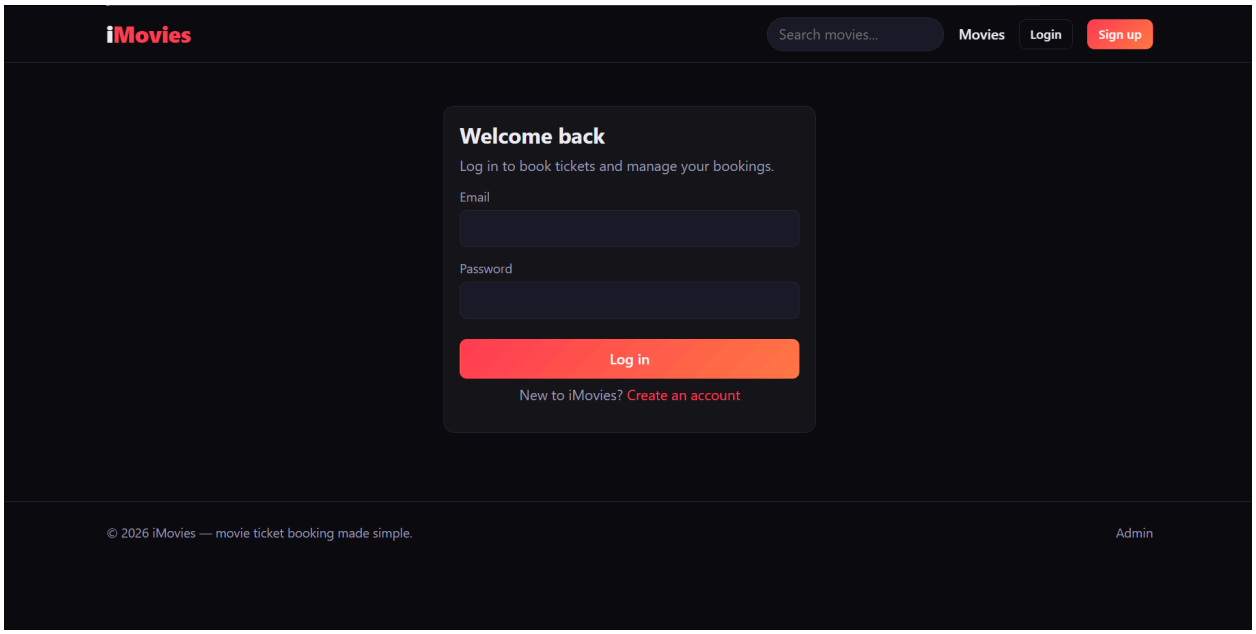
## 10. Testing

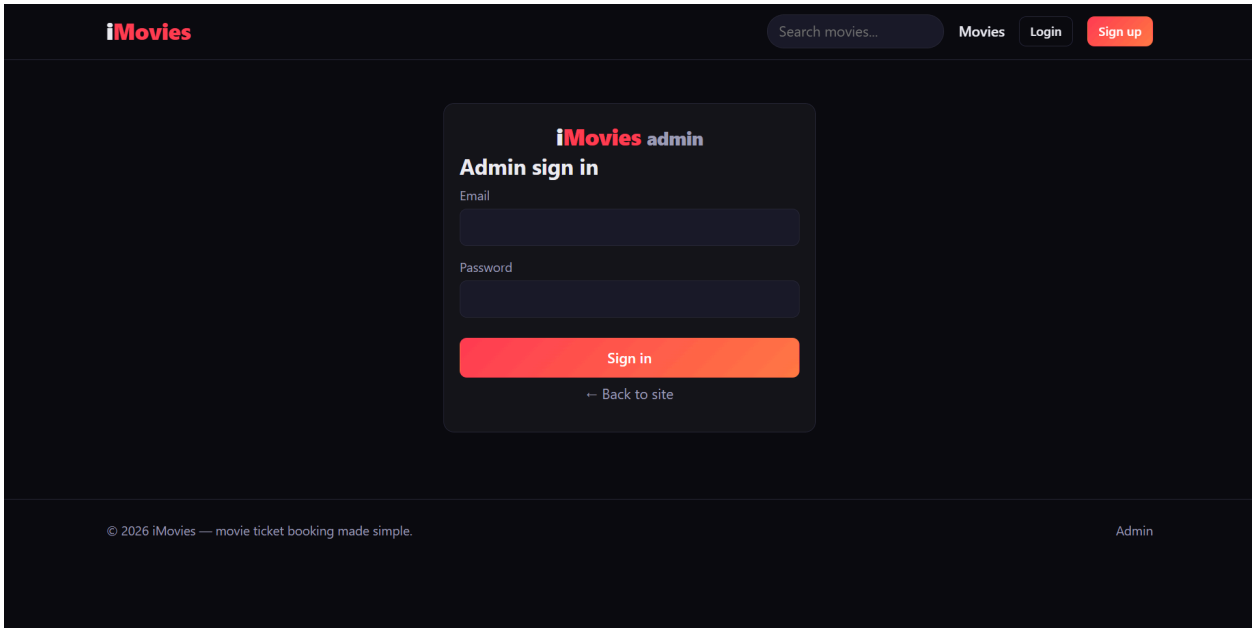
The project has no automated test suite yet; it was checked by hand against the main flows. Tools and checks available today:

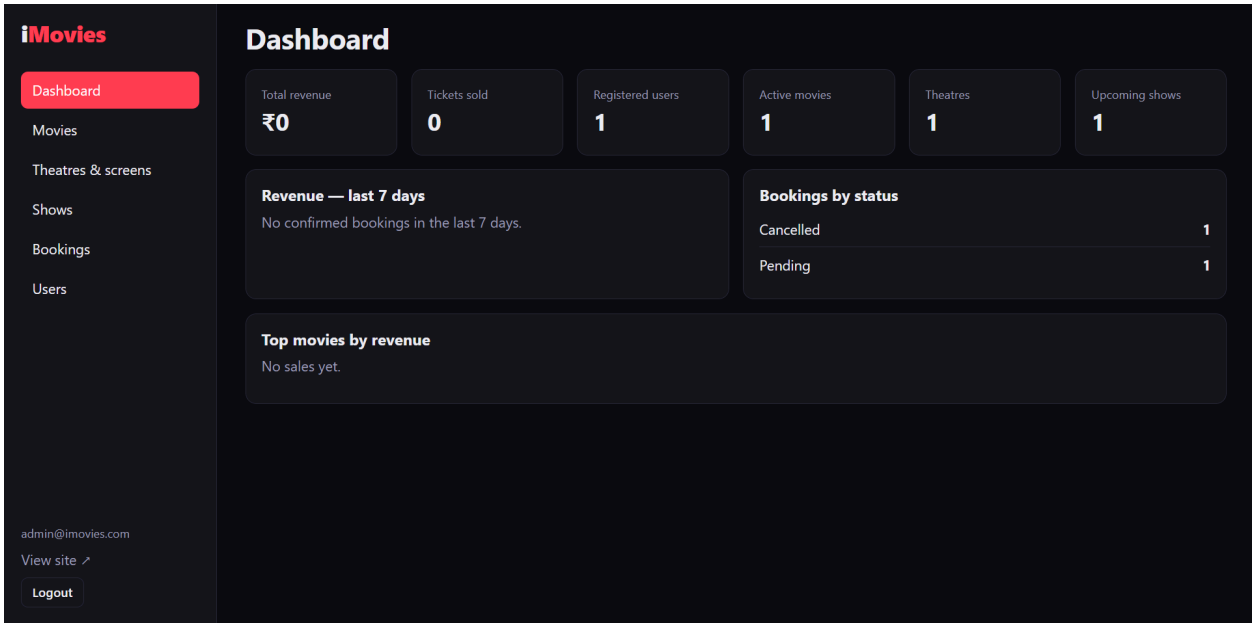
- **Manual end-to-end testing** of register, login, browse, seat hold, payment, ticket, cancel, review and favourites, plus every admin screen.
- **Seeded demo data** (`npm run seed`) gives a repeatable starting state.
- **Payment test cases:** a card ending `0000` is declined; invalid card numbers or UPI ids are rejected; net banking and wallet succeed.
- **API checks** with Postman or curl against `/api/health` and the endpoints in section 7.
- **ESLint** (`npm run lint` in `client/`) for static checks.

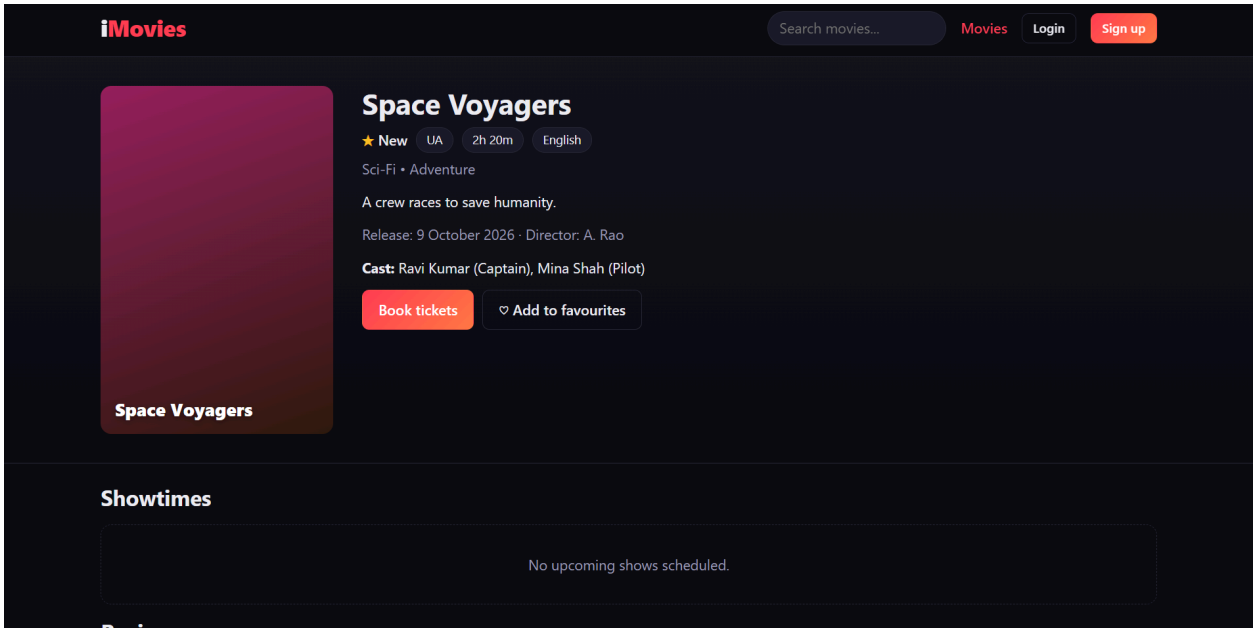
## 11. Screenshots or Demo

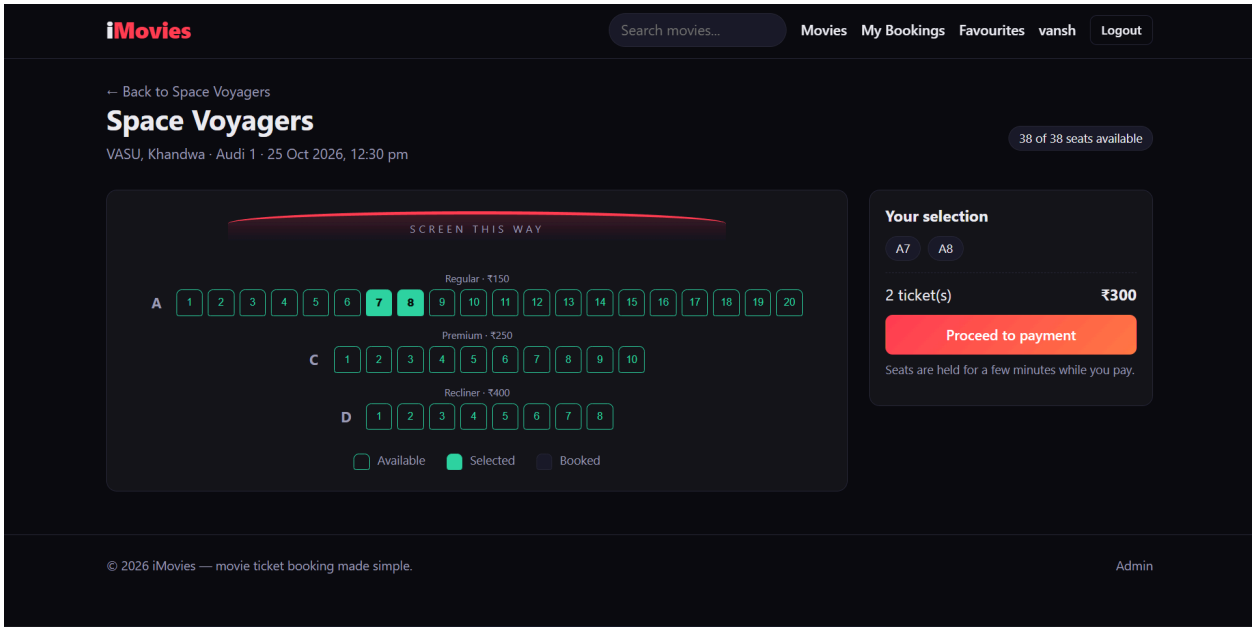


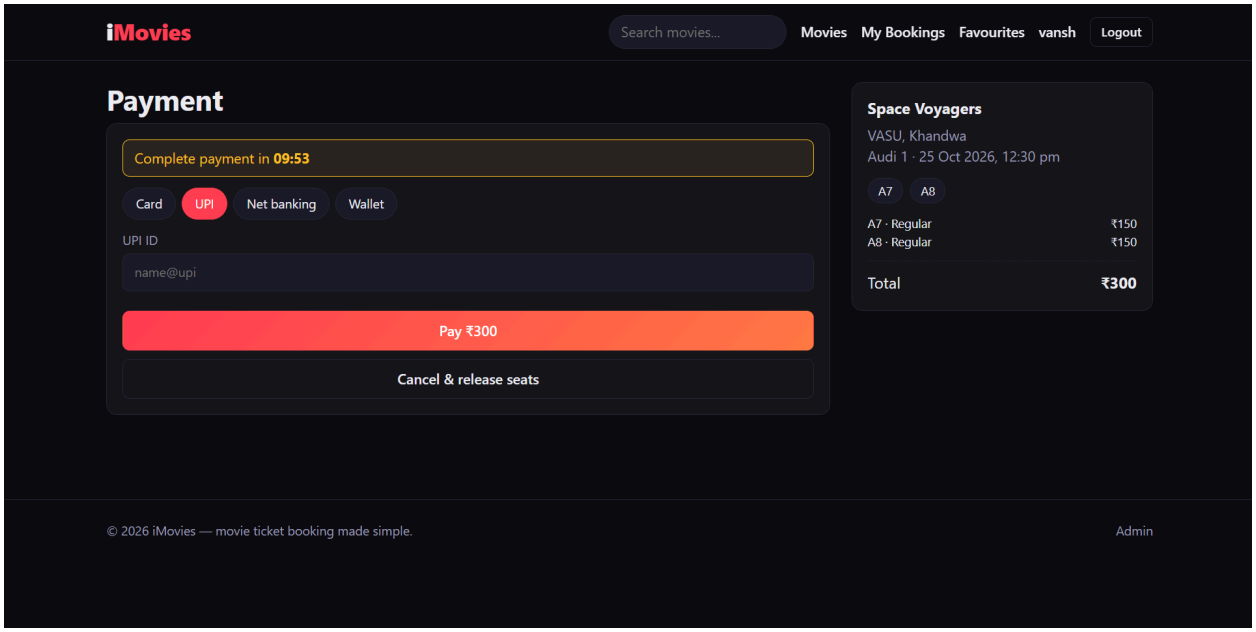



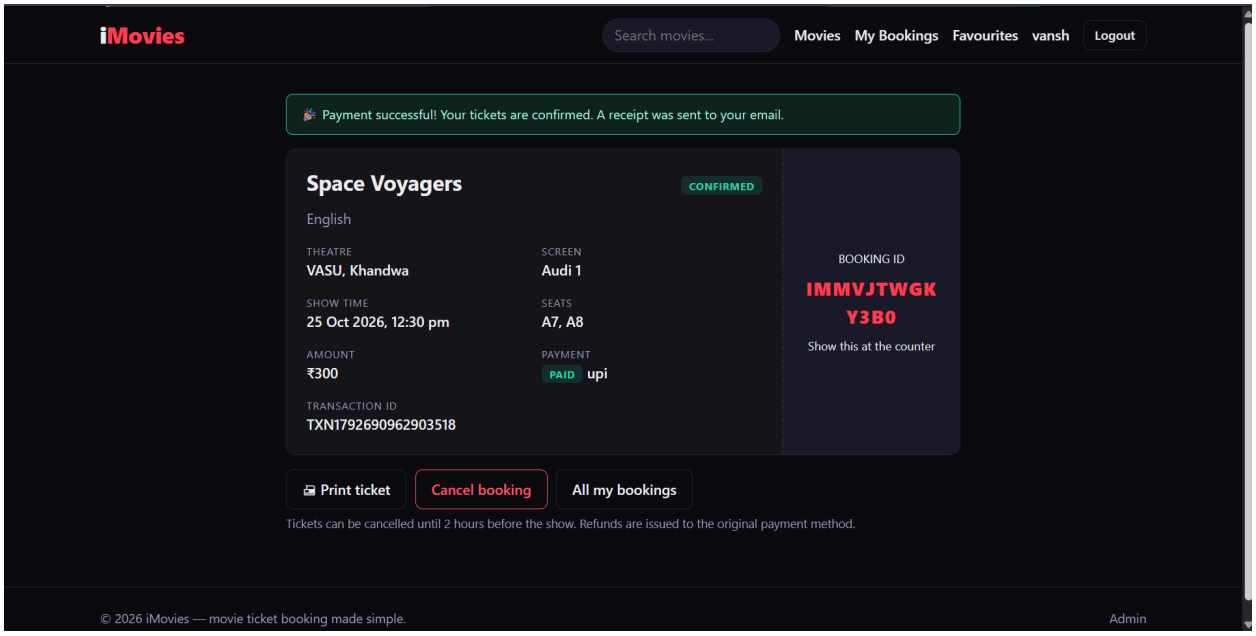

iMovies
Search movies...
Movies
Login
Sign up
iMovies admin
Admin sign in
Email
Password
Sign in
← Back to site
© 2026 iMovies — movie ticket booking made simple.
Admin


iMovies
Dashboard
Movies
Theatres & screens
Shows
Bookings
Users
admin@imovies.com
View site ↗
Logout
Dashboard
Total revenue
₹0
Tickets sold
0
Registered users
1
Active movies
1
Theatres
1
Upcoming shows
1
Revenue — last 7 days
No confirmed bookings in the last 7 days.
Bookings by status
Cancelled 1
Pending 1
Top movies by revenue
No sales yet.


iMovies
Search movies...
Movies
Login
Sign up
Space Voyagers
Space Voyagers
★ New
UA
2h 20m
English
Sci-Fi • Adventure
A crew races to save humanity.
Release: 9 October 2026 · Director: A. Rao
Cast: Ravi Kumar (Captain), Mina Shah (Pilot)
Book tickets
♡ Add to favourites
Showtimes
No upcoming shows scheduled.


iMovies
Search movies...
Movies
My Bookings
Favourites
vansh
Logout
← Back to Space Voyagers
Space Voyagers
VASU, Khandwa · Audi 1 · 25 Oct 2026, 12:30 pm
38 of 38 seats available
SCREEN THIS WAY
Regular · ₹150
A 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
Premium · ₹250
C 1 2 3 4 5 6 7 8 9 10
Recliner · ₹400
D 1 2 3 4 5 6 7 8
Available
Selected
Booked
Your selection
A7
A8
2 ticket(s)
₹300
Proceed to payment
Seats are held for a few minutes while you pay.
© 2026 iMovies — movie ticket booking made simple.
Admin




## 12. Known Issues

- Payments are simulated: no real gateway is connected and refunds are only marked as initiated.
- Emails are printed to the server console unless the `SMTP_*` variables are set.
- The seeded shows are scheduled one day ahead, so "No upcoming shows" appears later until an admin schedules new ones.
- Port 5000 may already be in use (for example macOS AirPlay); change `PORT` in `server/.env` and the proxy target in `client/vite.config.js`.
- The seeded admin password is public in the README and must be changed before any real deployment.
- No automated tests exist.

## 13. Future Enhancements

- Real payment gateway (for example Razorpay or Stripe) with actual refunds
- Automated unit and API tests, and CI
- SMS and email ticket delivery with a QR code
- Poster and image upload by admins instead of URLs
- Food and beverage add-ons, coupons and offers
- Movie recommendations based on favourites and past bookings
- Multi-language interface and a mobile app